

Chapter 3 Lab: Solving Problems by Searching

AIMA 4th edition, Sections 3.1–3.4 · Total: 50 points

Name _____

Show your traces, tables, and one-sentence justifications; partial credit is based on them. Problems 3 and 4 each have exactly one correct trace under the conventions below, so apply them precisely.

Conventions used throughout this lab

1. Successors are generated in **alphabetical order** of state name.
2. A node is **expanded** when it is popped from the frontier and its successors are generated.
3. **Breadth-first search** follows Figure 3.9: the goal test is applied when a child is **generated**, and reached is a set of states.
4. **Uniform-cost search** is best-first search with $f(n) = g(n)$ (Figure 3.7): the goal test is applied when a node is **popped**, and ties on g are broken alphabetically by state. When a cheaper path to an already-reached state is found, it replaces the reached entry and is added to the frontier; you may either delete the older frontier entry or leave it in place, since both give the same answers in this lab.
5. **Depth-first search** is the Figure 3.12 loop with no depth limit: a LIFO stack, the goal test when a node is popped, IS-CYCLE checked on the popped node (following the parent chain all the way up), and no reached table. Push children so that the alphabetically first child is popped first.
6. **Depth-limited search** with limit ℓ uses the same loop, with the root at depth 0. A popped node that is not a goal and has depth ℓ sets result to cutoff and is **not expanded**, whether or not it has successors.

Problem	Topic	Points	Score
1	Formulating a search problem	10	
2	State-space graph vs. search tree	6	
3	Tracing BFS, DFS, and UCS	14	
4	Depth-limited and iterative deepening search	8	
5	Bidirectional search	4	
6	True or false, with justification	8	
	Total	50	

Problem 1: Formulating a Search Problem (10 points)

A tide-pool survey robot works on a grid of 5 rows and 8 columns. Six of the 40 cells are rock and can never be entered, so 34 cells are passable. The robot starts on the launch cell, facing north. Six passable cells hold specimens, and the survey is complete when the robot has photographed all six specimens and is standing on the launch cell again.

The robot has four actions, and each costs 1:

- **Forward:** move one cell in the direction the robot is facing. Not applicable if that cell is rock or off the grid.
- **TurnLeft** and **TurnRight:** rotate 90 degrees in place.
- **Snap:** photograph the specimen on the current cell. Applicable only on a specimen cell.

The robot's software also logs the water temperature after every action and tracks battery charge, which is always more than enough to finish the survey.

- (a) [3 pts] Define a minimal state representation. List every state variable and its domain. Then name one piece of information from the description that you left out, and explain why it is safe to leave out.

- (b) [2 pts] How many states are in your state space? Show the factors you multiplied.

- (c) [2 pts] Write the goal test in terms of your state variables. Then write `RESULT(s, TurnLeft)` for a state in which the robot faces north.

- (d) [3 pts] Column 5 of the grid is a tidal channel. Its cells are passable (none of the six rock cells is in the channel), but the robot may move Forward into a channel cell only if it has taken fewer than 12 actions so far. Every action counts, including turns and Snap. What must you add to the state? Explain why a single Boolean such as *channelOpen* is not enough, and give the new number of states.

Problem 2: State-Space Graph vs. Search Tree (6 points)

Figure 2 shows a state space with initial state S. Each arrow is an action. In this problem, **ignore the goal test**: count every node the search tree would contain, including nodes for G.

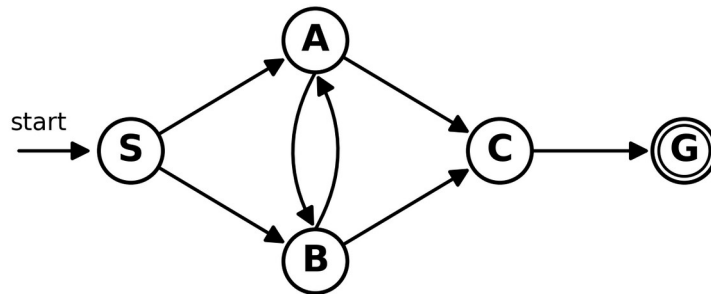


Figure 2. Directed state space for Problem 2.

- (a) [2 pts] Build the search tree with no repeated-state checking of any kind. How many nodes are at depth 2? How many are at depth 3? List the depth-3 nodes as paths written as state sequences, such as S-X-Y-Z.

- (b) [1 pt] Is this search tree finite or infinite? Name the feature of the graph responsible.

- (c) [2 pts] Now keep only nodes whose path contains no repeated state (cycle checking all the way up the parent chain). How many nodes are at depth 3? Is the tree finite now? If it is, how many nodes does it contain in total?

- (d) [1 pt] Cycle checking does not remove every redundant path. Name a state that still appears in more than one node of the tree from part (c), give two of its paths, and name the mechanism from Section 3.3.3 that would keep only one of them.

Problem 3: Tracing BFS, DFS, and UCS (14 points)

Figure 3 shows a directed state space. Edge labels are action costs. The initial state is S and the only goal state is G. Apply the conventions on page 1 exactly.

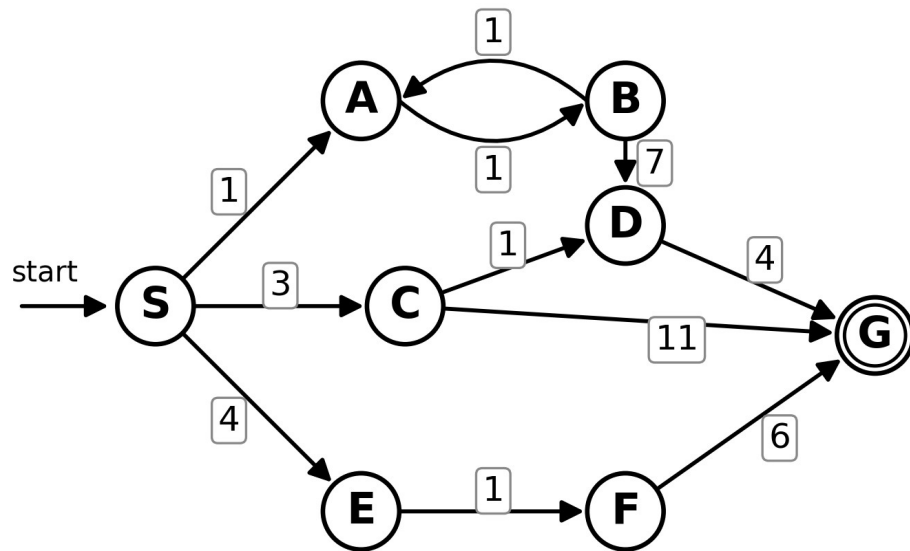


Figure 3. Weighted directed state space for Problem 3.

- (a) **[3 pts] Breadth-first search** (Figure 3.9). List the nodes in the order they are expanded. Give the returned path and its cost.

Expansion order

Returned path

Path cost

- (b) **[3 pts] Depth-first search** (Figure 3.12 loop, no depth limit). List every node in the order it is popped, and mark with an X any node that is discarded by the cycle check. Give the returned path and its cost.

Pop order

Returned path

Path cost

- (c) **[5 pts] Uniform-cost search.** Complete the table with one row per pop. Write frontier entries as state(g), and record every time a cheaper path replaces an entry in the reached table. Row 1 is done for you. Then give the returned path, its cost, and the pop number on which it is returned.

Pop #	Node popped (g)	Frontier after the pop	Reached-table replacement
1	S(0)	A(1), C(3), E(4)	none
2			
3			
4			
5			
6			
7			
8			
9			

Returned path

Path cost

Returned on pop #

- (d) **[1 pt]** Suppose UCS applied the goal test when a node is **generated**, as BFS does. Which path would it return, at what cost, and during which pop?

- (e) **[2 pts]** A teammate codes Figure 3.9 in Python with a `collections.deque` but writes `frontier.pop()` instead of `frontier.popleft()`. Children are still appended in alphabetical order, and the goal test and reached set are unchanged. What path and cost does the buggy code return? Is it the path from part (b)? Explain in one or two sentences.

Problem 4: Depth-Limited and Iterative Deepening Search (8 points)

Figure 4 shows a tree rooted at A. The goal state is N. Iterative deepening search calls depth-limited search with $\ell = 0, 1, 2, \dots$ until the result is not cutoff. Apply convention 6 on page 1.

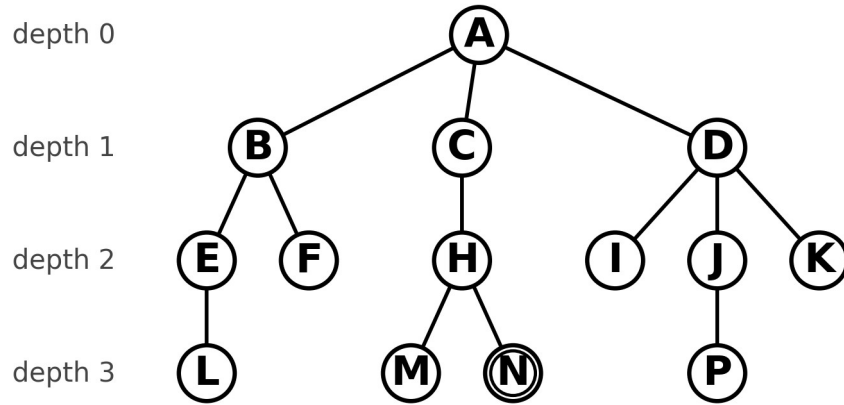


Figure 4. Search tree for Problem 4. The goal state N is drawn with a double circle.

- (a) [3 pts] Complete the table. For each limit, list every node in the order it is popped and the value that depth-limited search returns (solution, cutoff, or failure). For a solution, also give the path.

ℓ	Nodes popped, in order	Returns
0		
1		
2		
3		

- (b) [2 pts] Now suppose that no state in the tree is a goal. What does depth-limited search return with $\ell = 3$, and why? On which iteration (value of ℓ) does iterative deepening stop, and what does it return?

- (c) [3 pts] Section 3.4.4 compares the number of nodes generated by iterative deepening and by breadth-first search on a uniform tree with the solution at depth d (the root is not counted). For $b = 3$ and $d = 4$, compute $N(\text{IDS})$, $N(\text{BFS})$, and the percentage overhead of IDS. Lecture showed about 11 percent overhead for $b = 10$, $d = 5$. In one sentence, explain why the overhead differs.

Problem 5: Bidirectional Search (4 points)

- (a) [2 pts] For $b = 6$ and $d = 8$, compute b^d and $b^{d/2} + b^{d/2}$. By what factor does bidirectional search reduce the node count?

- (b) [2 pts] For each problem, state whether bidirectional search (Figure 3.14) can be applied directly, and name the requirement from Section 3.4.5 that is met or violated.

- (i) Finding a route from Arad to Bucharest on the Romania map.
- (ii) Knuth's "4" problem with goal 5, using the actions factorial, square root, and floor.

Problem 6: True or False (8 points)

Circle T or F, and justify each answer in one sentence. One point each.

1. **T F** Breadth-first search is cost-optimal on any problem in which every action has the same cost.

2. **T F** When depth-limited search returns cutoff, the problem has no solution.

3. **T F** Iterative deepening search needs $O(b^d)$ memory, because it eventually generates every node that breadth-first search generates.

4. **T F** Tree-like depth-first search with no cycle checking is complete on every finite state space.

5. **T F** Depth-first search that keeps a reached table is complete on every finite state space.

6. **T F** Breadth-first search can solve a problem with an infinite state space, as long as the branching factor is finite and a solution exists.

7. **T F** In bidirectional best-first search (Figure 3.14), the forward and backward searches take turns, each expanding one node per iteration.

8. **T F** If every action cost is at least $\epsilon > 0$, uniform-cost search expands every state whose cheapest path cost is less than C^* before it returns a solution of cost C^* .